

CS330 Operating Systems and Lab

Process & Thread

Spring 2026

KAIST

Instructor : Insik Shin

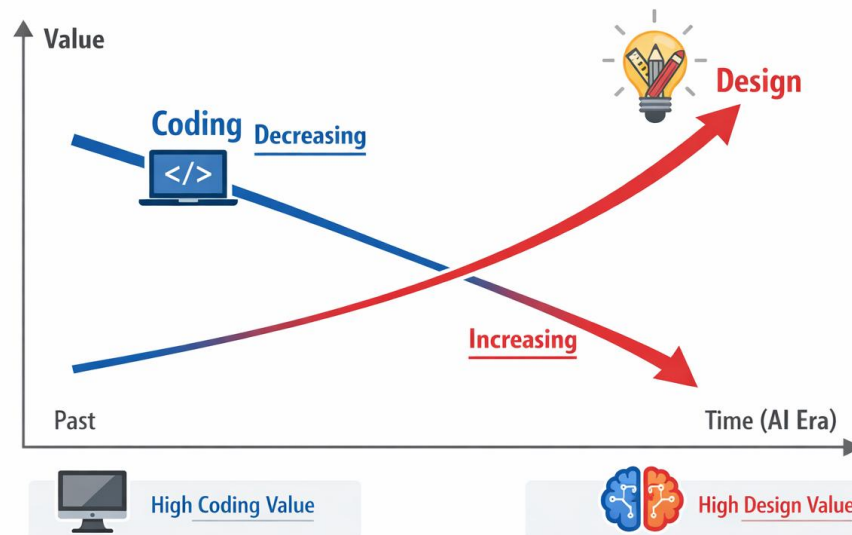
Key Goals

A key goal in this course is
to get an idea

how to design operating systems

Coding is becoming cheaper.

Design is becoming more valuable.



Bottleneck



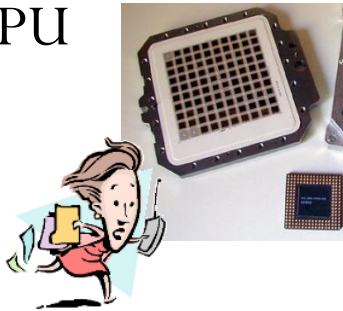
- A phenomenon where the performance of an entire system is limited by one or more components/resources
- System designers will try to avoid bottlenecks
 - try to locate and tune existing bottlenecks



The I/O Bottleneck

- In old days, memory and storage were faster than the processor
 - they were waiting for CPU to finish computation to feed data

CPU



RAM

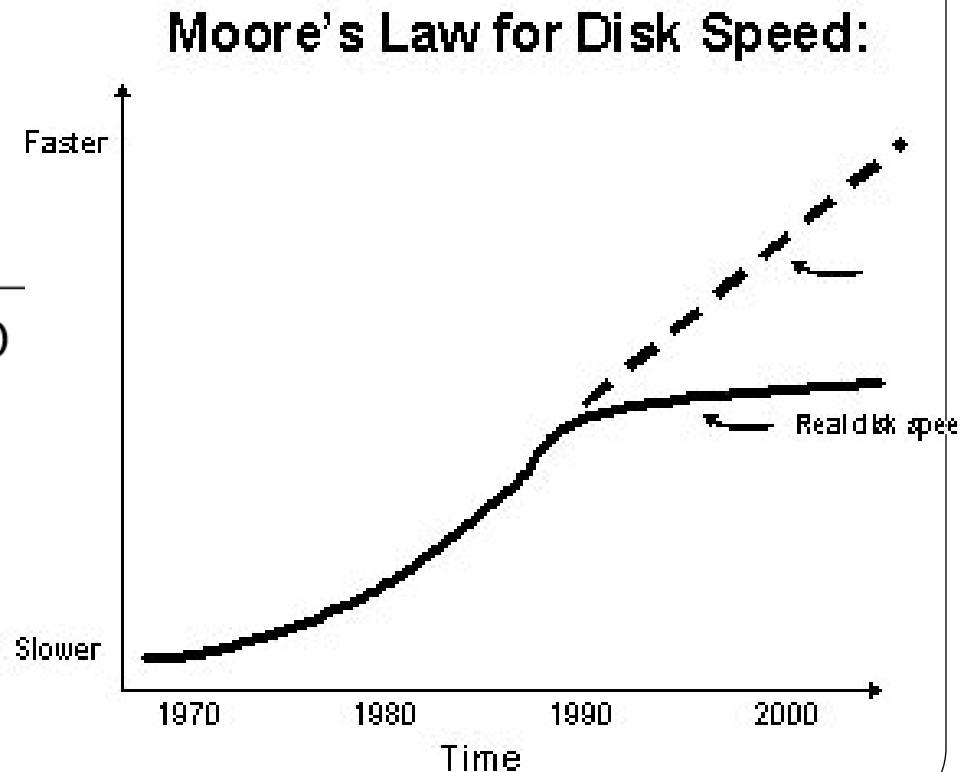
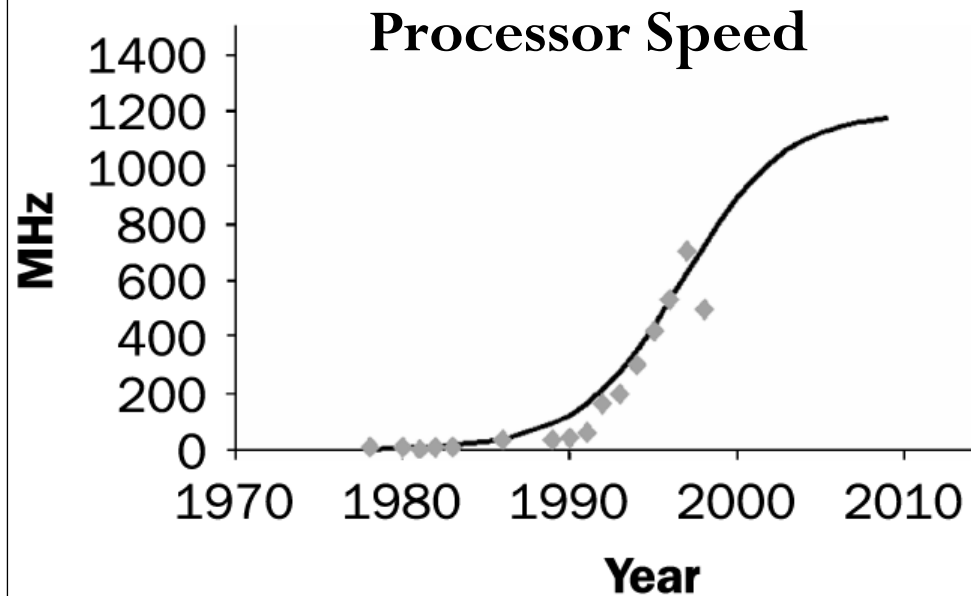


Storage



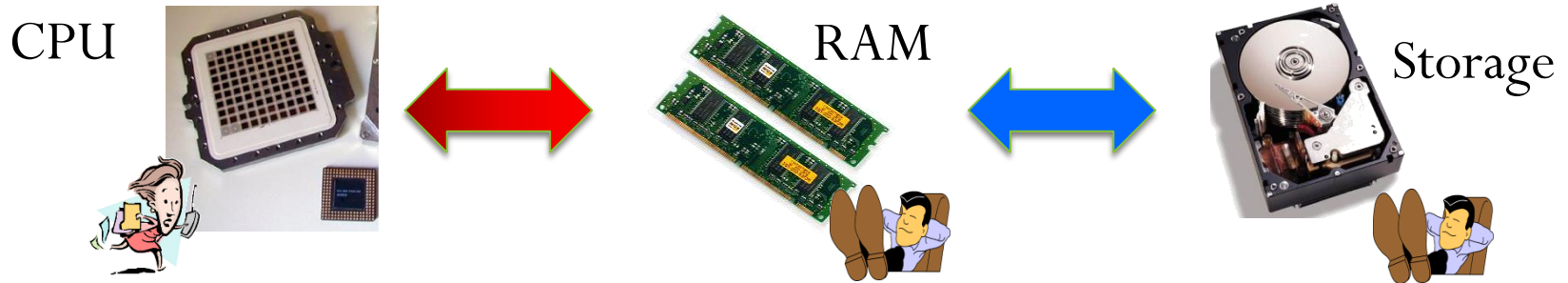
The I/O Bottleneck

- Over the years, processor speed was improved more than that of memory and storage

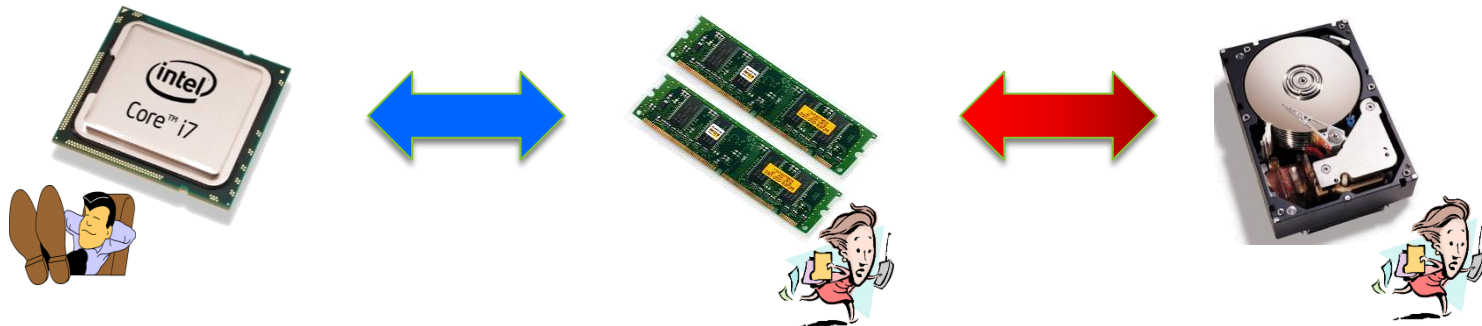


The I/O Bottleneck

- In old days, memory and storage were faster than the processor
 - they were waiting for CPU to finish computation to feed data



- Over the years, processor speed was improved more than that of memory and storage
- Nowadays, **the processor is waiting** for memory and storage drive to feed data

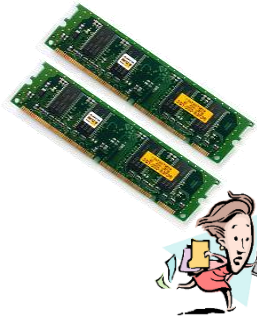


How to Design OS ?

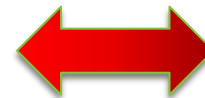
- How to design OS to overcome the I/O bottleneck?
 - Multi-programming
 - Bigger cache & better cache management algorithms



Multicores
4 x 3.5 GHz



Memory bandwidth
21GB/s



Disk (SATA)
6GB/s

Uniprogramming vs. Multiprogramming

- Uniprogramming: *one program (application) at a time*
 - MS/DOS, early Macintosh, batch processing
 - Easier for operating system builder
 - Get rid of concurrency (only one program accessing resources!)
 - Does this make sense for personal computers?
- Multiprogramming: *more than one programs at a time*
 - Multics, UNIX/Linux, OS/2, Windows NT – 7, Mac OS X

Solution Design

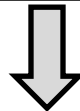
Goals:

Addressing the I/O bottleneck



Solution:

- Multiprogramming: Run multiple applications concurrently



Any challenges?

Challenges of Multiprogramming

- Each application wants to own the machine → **virtual machine abstraction**
- Applications compete with each other for resources
 - Need to arbitrate access to shared resources → **concurrency**
 - Need to protect applications from each other → **protection**
- Applications need to communicate/cooperate with each other → **concurrency**

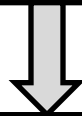
We will cover the above issues in the next several lectures

Today's Lecture - Objectives

- How do we provide multiprogramming?
 - What are **processes**?
 - How are they related to **threads** and **address spaces**?

Goals:

- Multiprogramming: Run multiple applications concurrently
- Protection: Don't want a bad application to crash system!



Solution:

Process: unit of execution and allocation

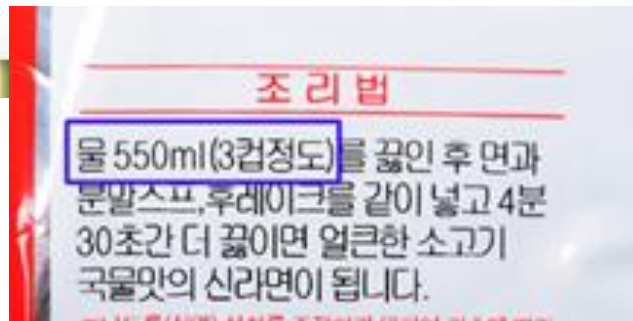
- Virtual Machine abstraction: gives an illusion that it owns a machine (i.e., CPU, Memory, and IO device multiplexing)

- Recipe

Warm Potato & Green Bean Salad

12-14 small new potatoes	2 T. red wine vinegar
1 c. green beans halved	1/2 t. salt
1/2 sm. red onion, slivered	1/8 t. black pepper
1/2 red pepper, chopped	2 t. fresh parsley, chopped
1 garlic clove, grated	1/8 t. each basil, oregano & thyme
3 T. olive oil	

Cook unpeeled potatoes covered in water until tender, about 15 minutes, adding green beans to pan for last 5 minutes to barely cook. Drain and place in salad bowl. Add onion and pepper to bowl. Mix remaining dressing ingredients and pour over potato mix tossing gently to coat. Let sit at room temperature about 1 hour before serving. Serves 4.



Cooking



Single program (chef)



Multiple programs (chefs)



two chefs can be either
independent or collaborative

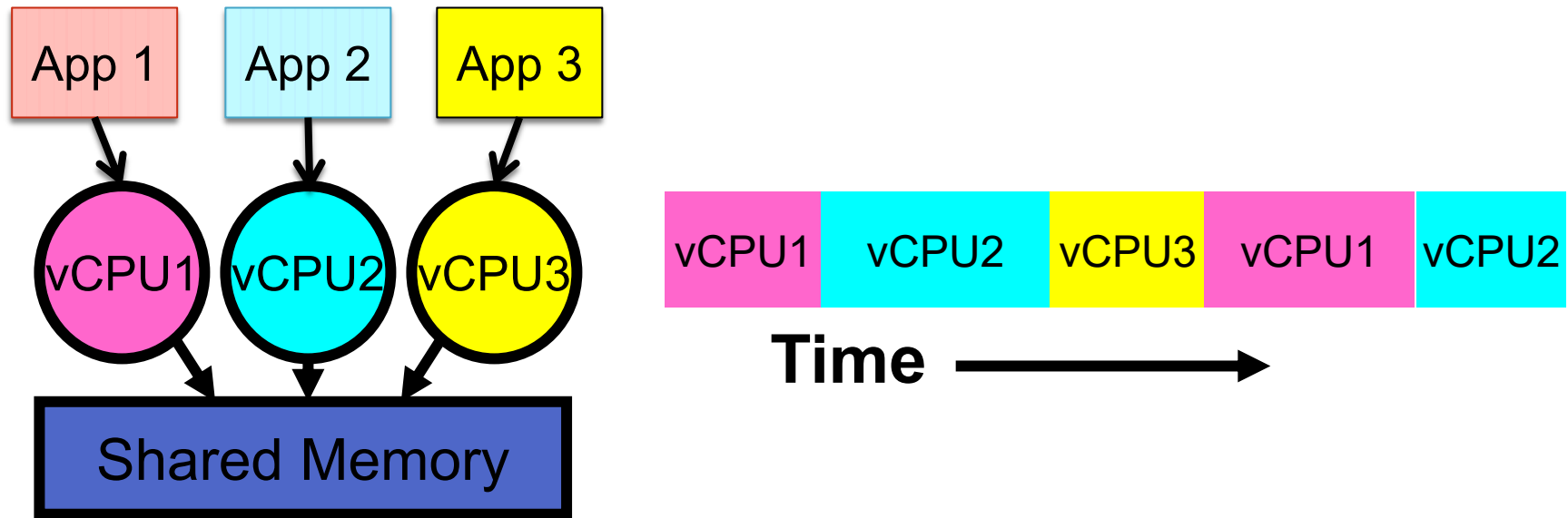


Illusion of using the kitchen
(machine) alone





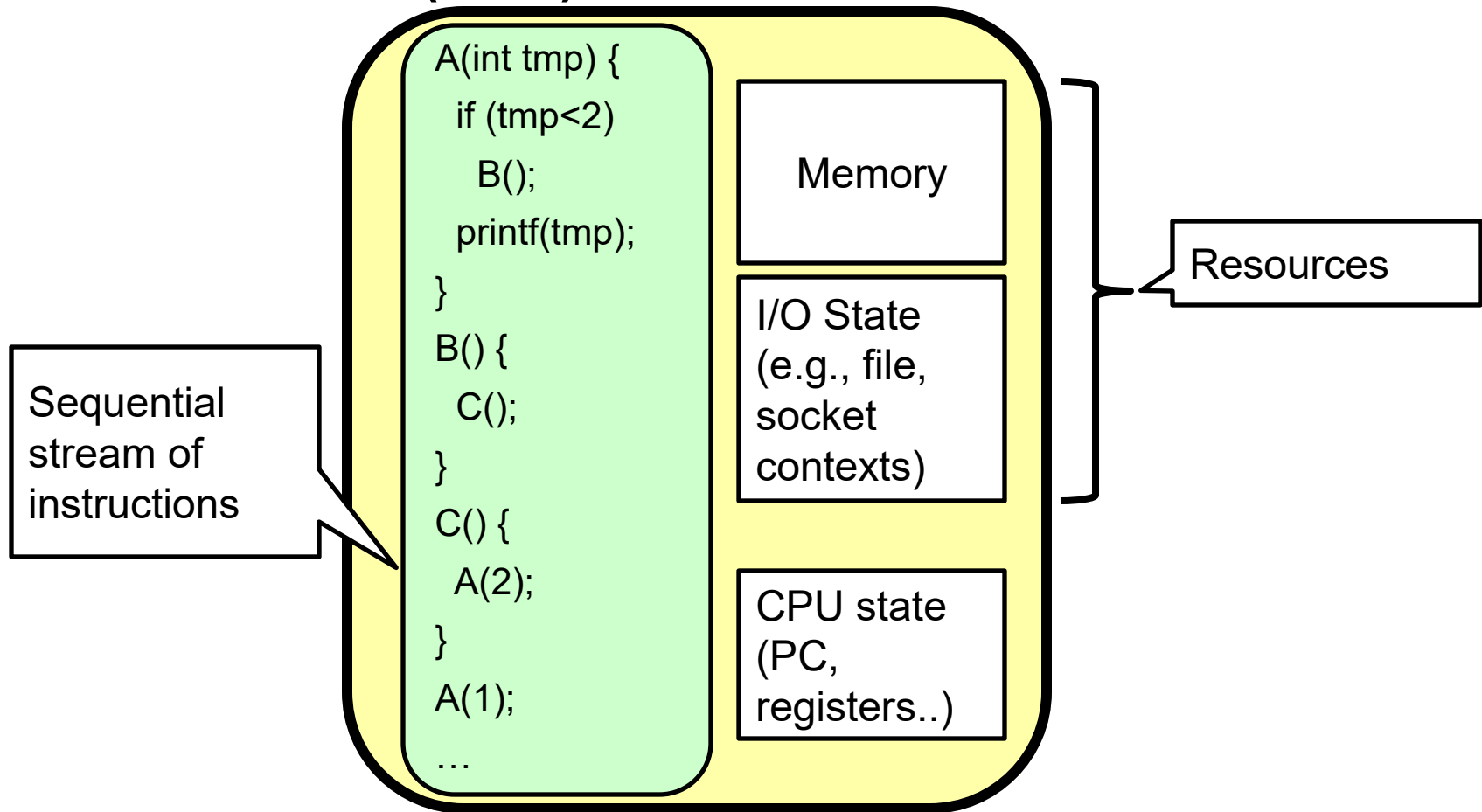
How can we give the illusion of multiple processors?



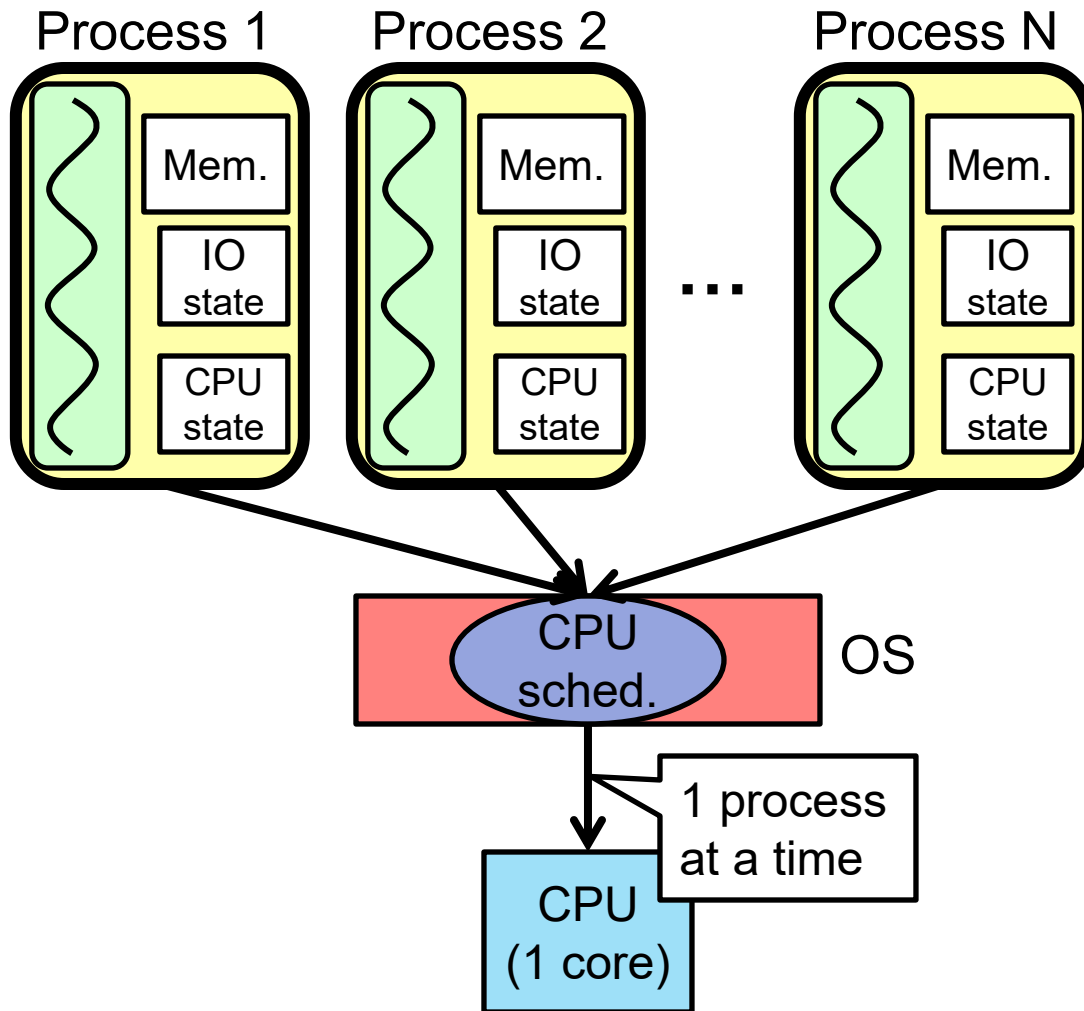
- Assume a single processor. How do we provide the illusion of multiple processors?
 - Multiplex in time!

Process

(Unix) Process

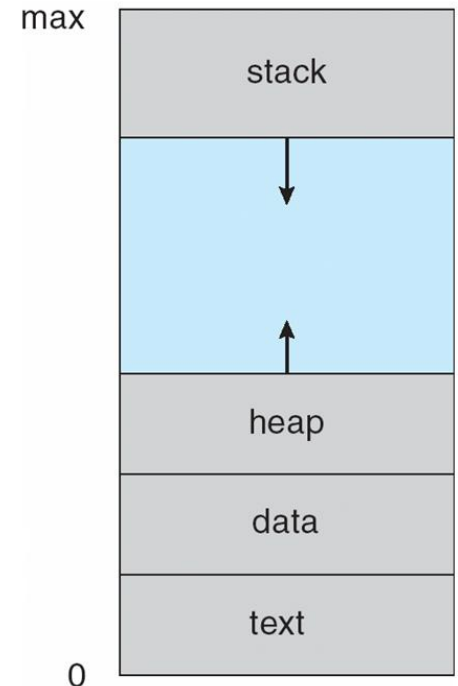


Processes



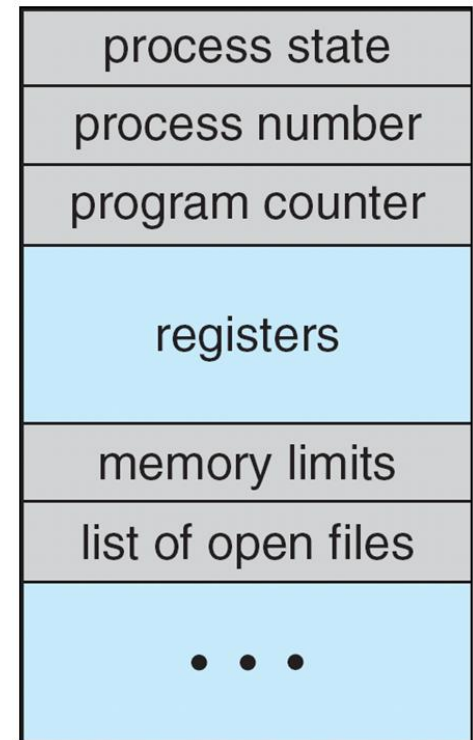
Viewpoint of Process

- Each process has its own view of machine
 - Its own address space
 - Its own open files
 - Its own virtual CPU (through multitasking)
- `*(char *) 0x8000` different in P1 & P2
- Greatly simplifies programming model
 - gcc does not care that `explorer` is running



Process Control Block (PCB)

- The current state of process is stored in a process control block (PCB):
 - This is a “snapshot” of the execution and protection environment
 - Only one PCB active at a time
 - OS keeps data structure for each process
- Information necessary to run
 - Program counter, CPU registers
 - Process state, scheduling info.
 - Memory management info.
 - I/O status info.
 - Credentials (user/group ID),
 - Accounting statistics

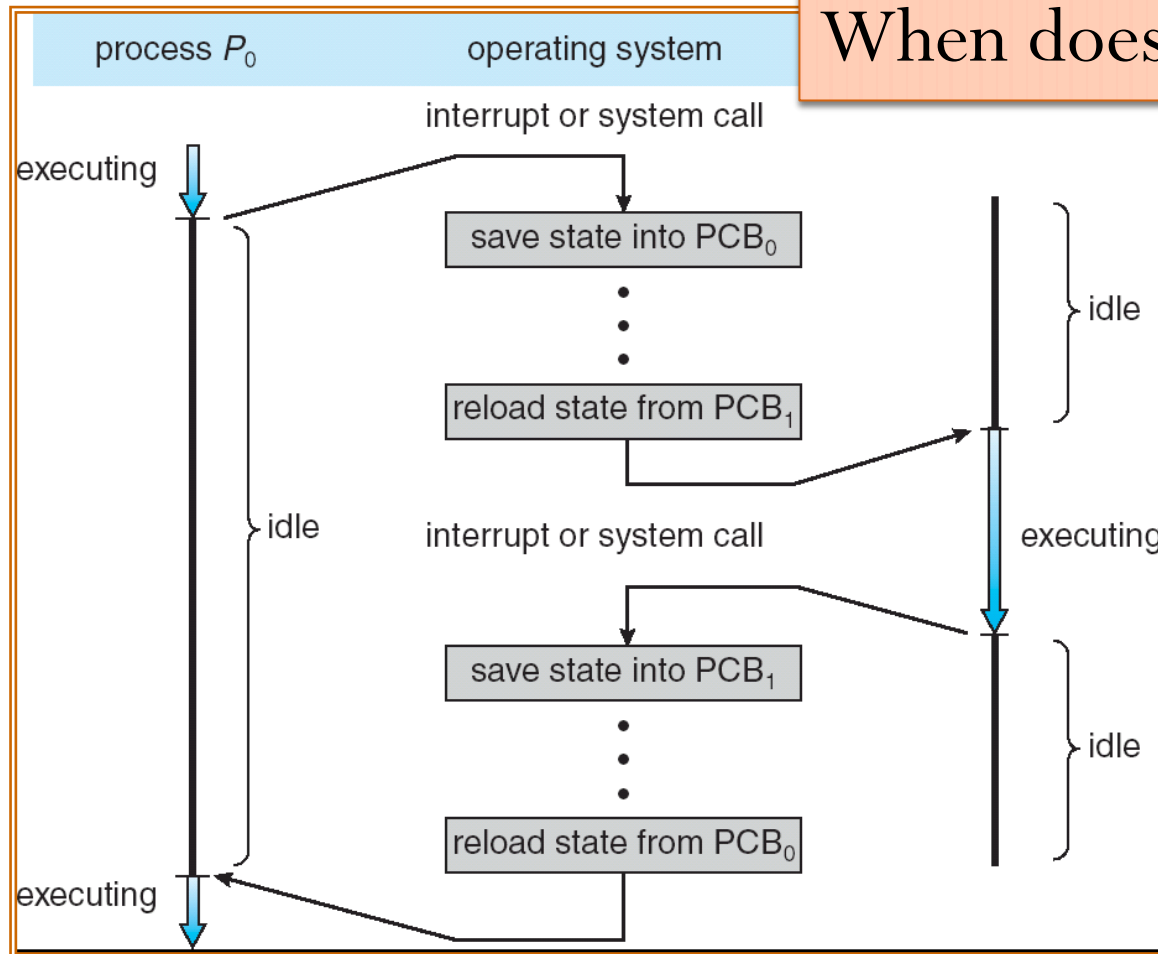


Process Control Block (Detailed)

Process Management	Memory Management	File Management
Registers Program counter Program status word Stack pointer Process state Priority Scheduling parameters Process group Parent process Signals Time when process started CPU time used Children's CPU time Time of next alarm	Pointer to text segment Pointer to data segment Pointer to stack segment Pointer to heap segment	Root directory Working directory File descriptors User ID Group ID

What is PCB for?

CPU Switch from Process to Process



When does it happen?

- This is also called a “context switch”
- Code executed in kernel above is overhead
 - Overhead sets minimum practical switching time

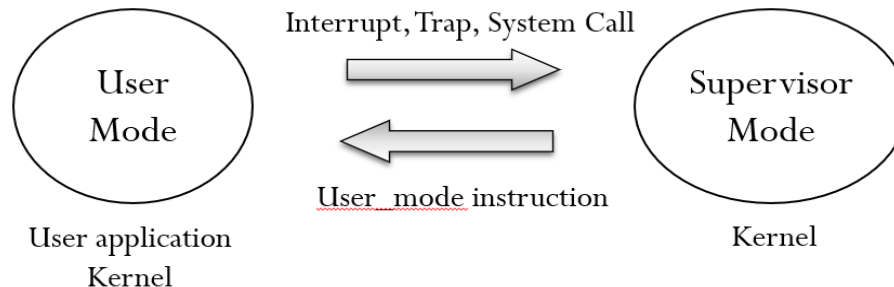
Preemption

- Process can be preempted when kernel gets control

Life Cycle

Side Slide from Lecture #2

- Schema of system operation
 - System powered on; in supervisor mode
 - System boots, kernel initializes, still in supervisor mode
 - System enters user mode to run user programs
- Should be there a user mode instruction to enter supervisor mode ?
- How can supervisor mode be re-entered?



- Differences, similarities btw. interrupts, traps, system calls?

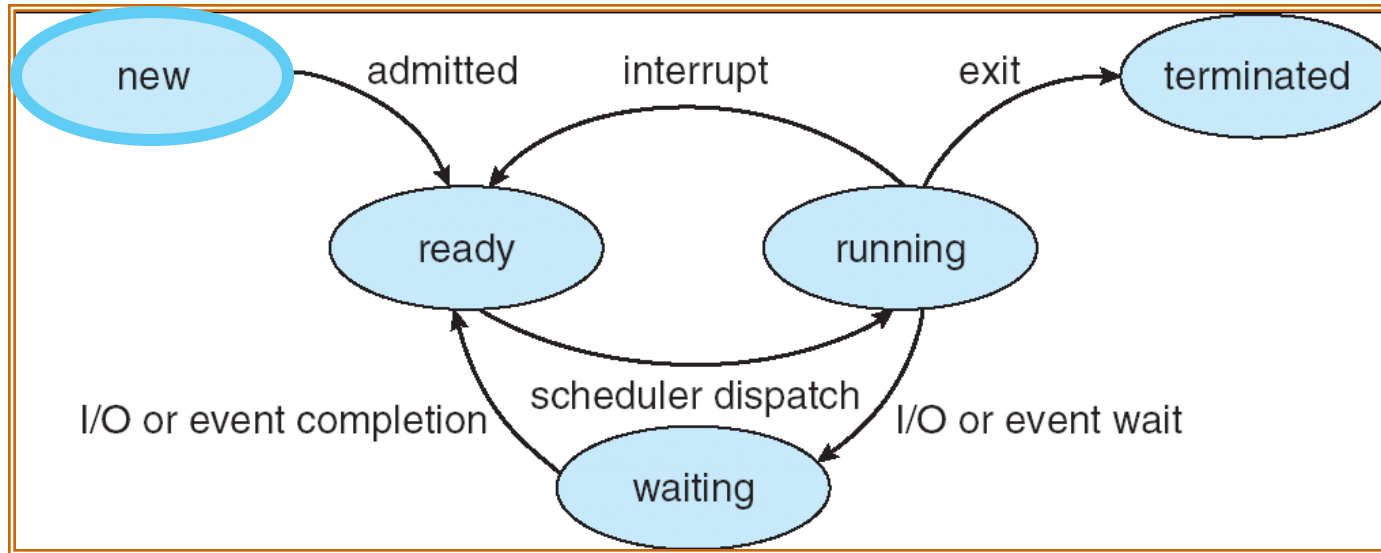
Preemption

- Process can be preempted when kernel gets control
 - Running proc. can pass control to kernel
 - system call, page fault, illegal instruction, etc
 - may put current proc. to sleep – e.g., read from disk
 - may make other proc. runnable – e.g., fork
 - Periodic timer interrupt
 - if running proc. used up quantum, schedule another
 - Device interrupt
 - disk request completed, or packet arrived on network
 - previously waiting process becomes runnable
 - schedule if higher priority process is available for execution

Context Switch

- Context switch
 - When CPU switches to another process, the system must save the state of the old process and load the saved state for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead
 - Typical speeds are a few milliseconds
 - the system does no useful work while switching
 - Time dependent on hardware support
 - Special instruction, multiple sets of registers, etc

Lifecycle of a Process



- As a process executes, it changes state:
 - **new**: The process is being created

What does it take to create a process?

- Must construct new PCB
 - Inexpensive
- Must create new address space
 - Set up new page tables for address space
 - More expensive
- Copy data from parent process? (Unix `fork()`)
 - Semantics of Unix `fork()` are that the child process gets a complete copy of the parent memory and I/O state
 - Originally *very* expensive
 - Much less expensive with “copy on write”
- Copy I/O state (file handles, etc)
 - Medium expense

Process

Side Note

Memory Space



SP



Stack

Heap

Static Data

PC



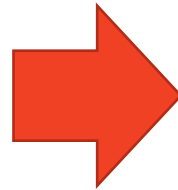
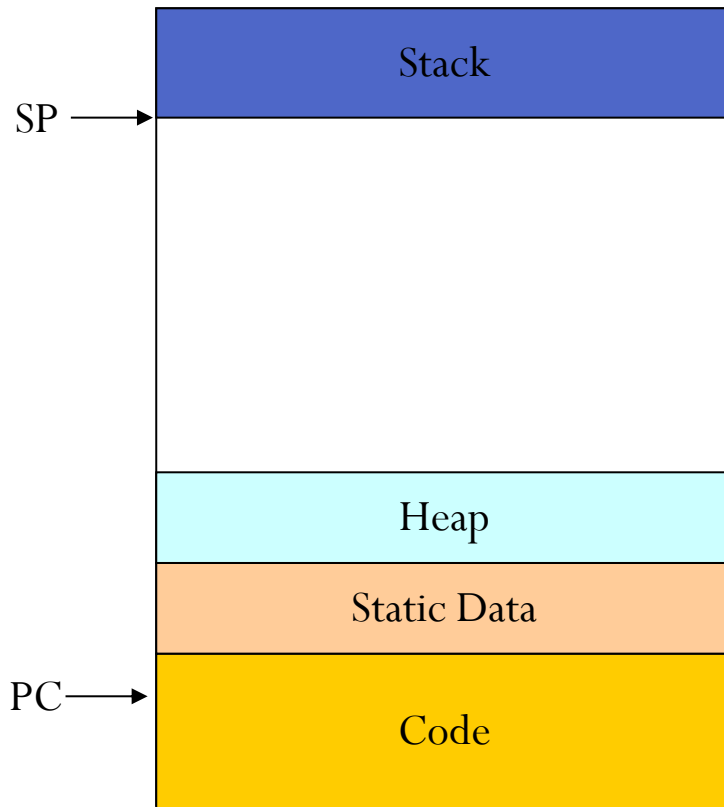
Code

- Execution context
 - Program counter (PC)
 - Stack pointer (SP)
 - Data registers
- Code, Data, Stack

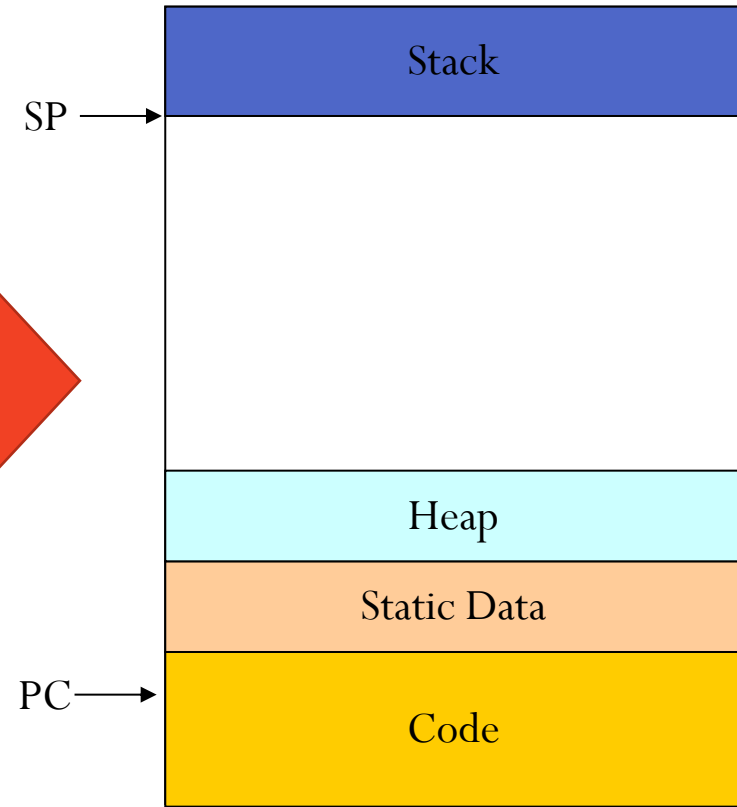
Process Creation

Side Note

Memory Space for **Parent**



Memory Space for **Child**



Process Creation : UNIX

```
int fork()
```

- creates a copy of itself, “child process”
 - creates a new PCB & a new address space
 - initializes the address space with a copy of the entire contents of the parent’s address space
 - Initializes the kernel resources to point the parent’s resources (e.g., open files, environment)
- returns the child’s PID to the parent, and zero to the child

C Program Forking Separate Process

```
int main()
{
1:  int pid;

2:  pid = fork(); /* fork another process */
3:  if (pid < 0) { /* error occurred */
4:      fprintf(stderr, "Fork Failed"); exit(-1);
5:  }
6:  else if (pid == 0) { /* child process */
7:      printf ("Child: I am %d. My parent is %d.\n", getpid(), getppid());
8:  }
9:  else { /* parent process */
10:      printf ("Parent: I am %d. My child is %d\n", getpid(), pid);
11:  }
}
```

fork(): Example Output

% ./a.out

Child: I am 31101. My parent is 31100.

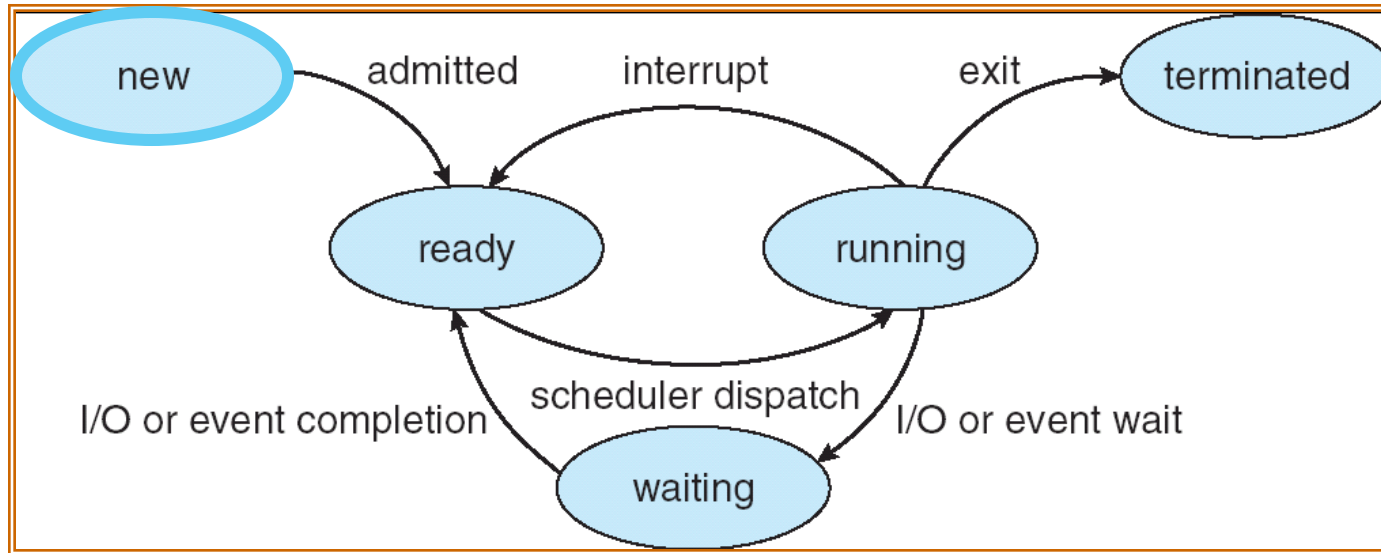
Parent: I am 31100. My child is 31101.

% ./a.out

Parent: I am 31102. My child is 31103.

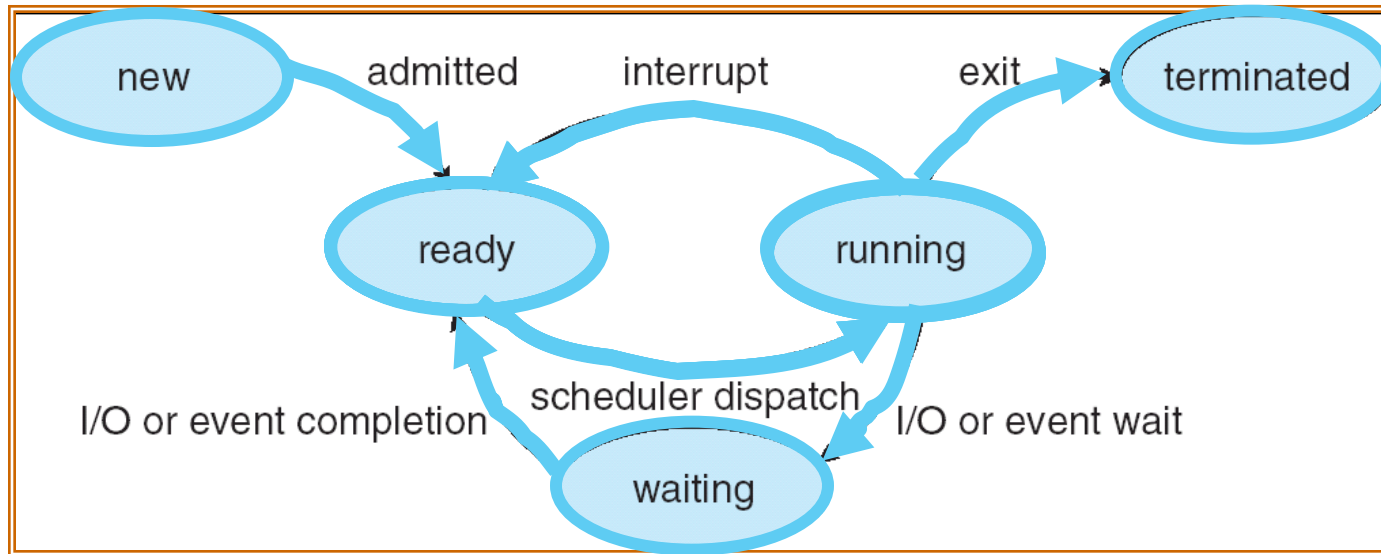
Child: I am 31103. My parent is 31102.

Lifecycle of a Process



- As a process executes, it changes state:
 - **new**: The process is being created

Lifecycle of a Process

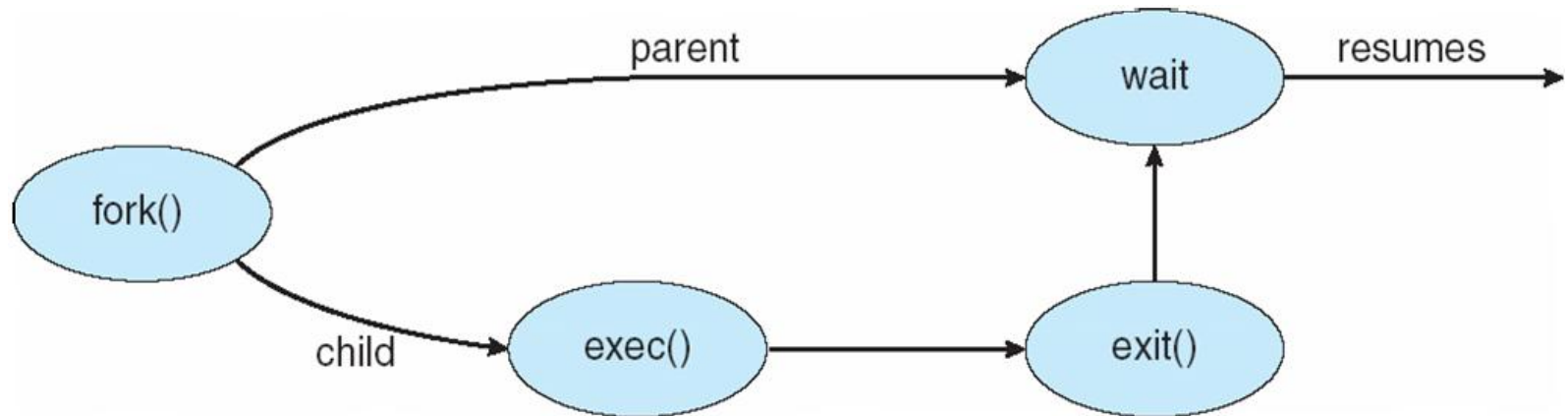


- As a process executes, it changes state:
 - **new**: The process is being created
 - **ready**: The process is waiting to run
 - **running**: Instructions are being executed
 - **waiting**: Process waiting for some event to occur
 - **terminated**: The process has finished execution

Process Termination

- Process executes last statement and asks the operating system to delete it (**exit**)
 - Output status data (in PCB) for parent (via **wait**)
 - Process' resources are deallocated by OS
- Parent may terminate execution of child processes (**abort**)
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - If parent is exiting
 - Some operating system do not allow child to continue if its parent terminates
 - All children terminated - **cascading termination**

Process Creation



Note on UNIX Process Termination

- What happens if child exits before parent?
 - Parent **must** call wait() to “harvest” child. OS will notify parent about child’s termination
 - If not, child process becomes a **zombie process**
- What happens if parent exits before child?
 - **Orphaned processes**
 - Re-parented to process 1, the **init** process
- How can a parent process be notified the termination of a child process in an asynchronous way?
 - Using signal handlers: Parent can use the “SIGCHLD” signal
 - Using “waitpid()”: it works in a non-blocking mode

Note on UNIX Process Termination

```
while (1) {  
    write (1, "$ ", 2);  
    parse_cmd (command, args); // parse user input  
    switch(pid = fork ()) {  
        case -1: perror ("fork"); break;  
        case 0: // child  
            execv (command, args, 0); break;  
        default: // parent  
            wait (0); break; // wait for child to terminate  
    }  
}
```

Multiprogramming & Process

- How do we provide multiprogramming?
 - What are **processes**?
 - How are they related to **threads** and **address spaces**?

Goals:

- Multiprogramming: Run multiple applications concurrently
- Protection: Don't want a bad application to crash system!



Solution:

Process: unit of execution and allocation

- Virtual Machine abstraction: give process illusion it owns machine (i.e., CPU, Memory, and IO device multiplexing)



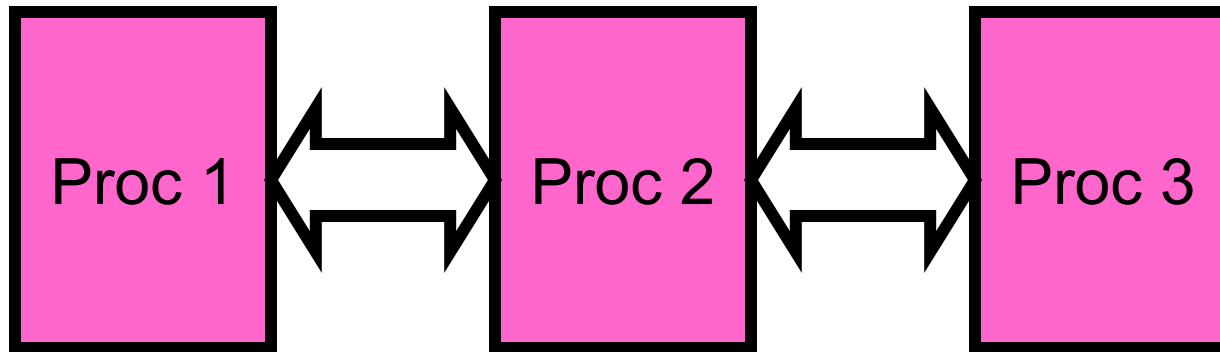
Challenge:

- Any challenges ?

Challenges of Multiprogramming

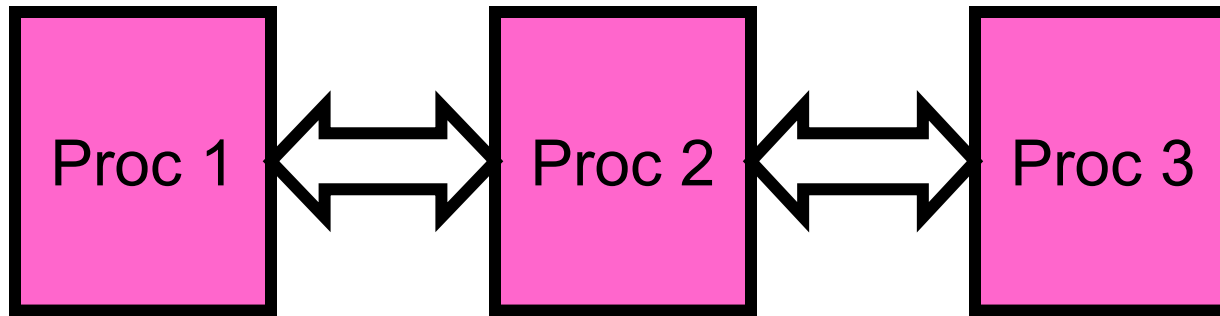
- Applications **compete** with each other for resources
 - Need to arbitrate access to shared resources → **concurrency (scheduling & synchronization)**
 - Need to protect applications from each other → **protection**
- Applications need to **communicate/cooperate** with each other → **concurrency**

Cooperating Processes



- Multiple processes collaborate on a task
any examples ?

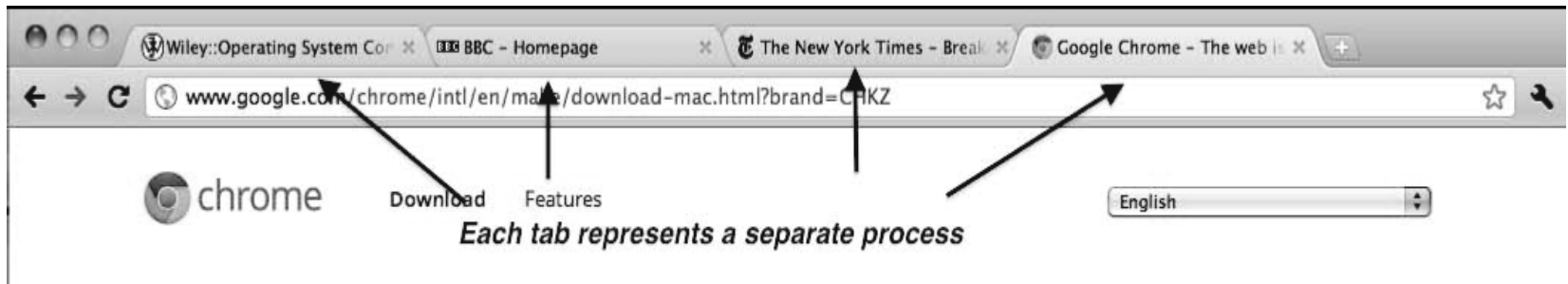
Cooperating Processes



- Multiple processes collaborate on a task
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

Multiprocess Architecture – Chrome Browser

- Many web browsers ran as single process (some still do)
 - If one web site causes trouble, entire browser can hang or crash
- Google Chrome Browser is multiprocess with 3 categories
 - **Browser** process manages user interface, disk and network I/O
 - **Renderer** process renders web pages, deals with HTML, Javascript, new one for each website opened
 - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
 - **Plug-in** process for each type of plug-in

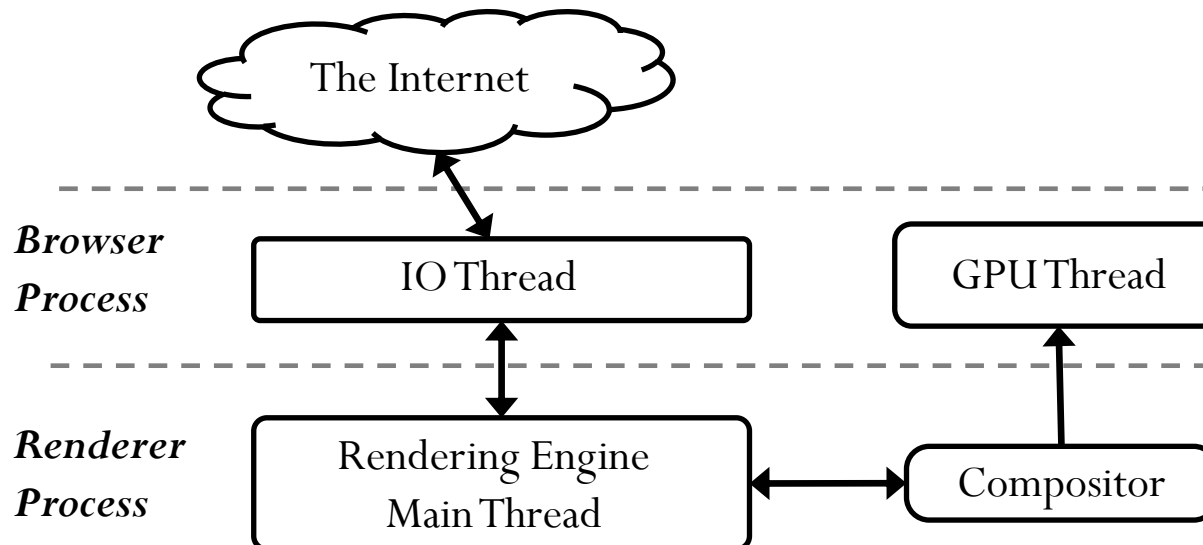
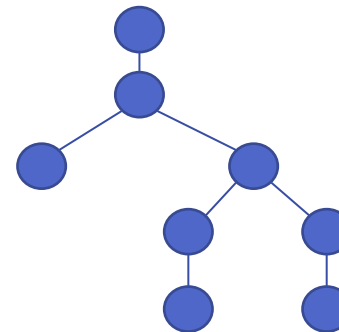
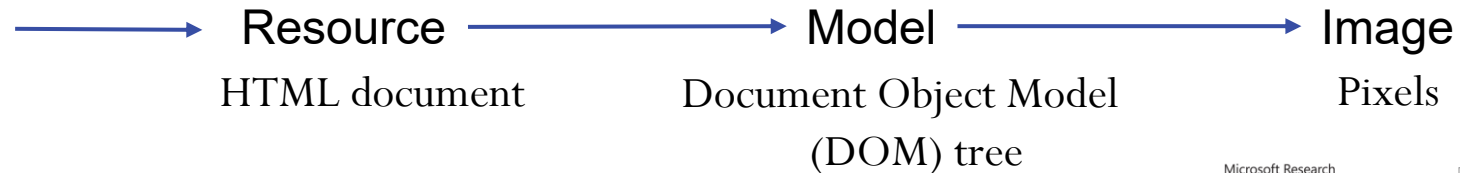


Browser Workflow

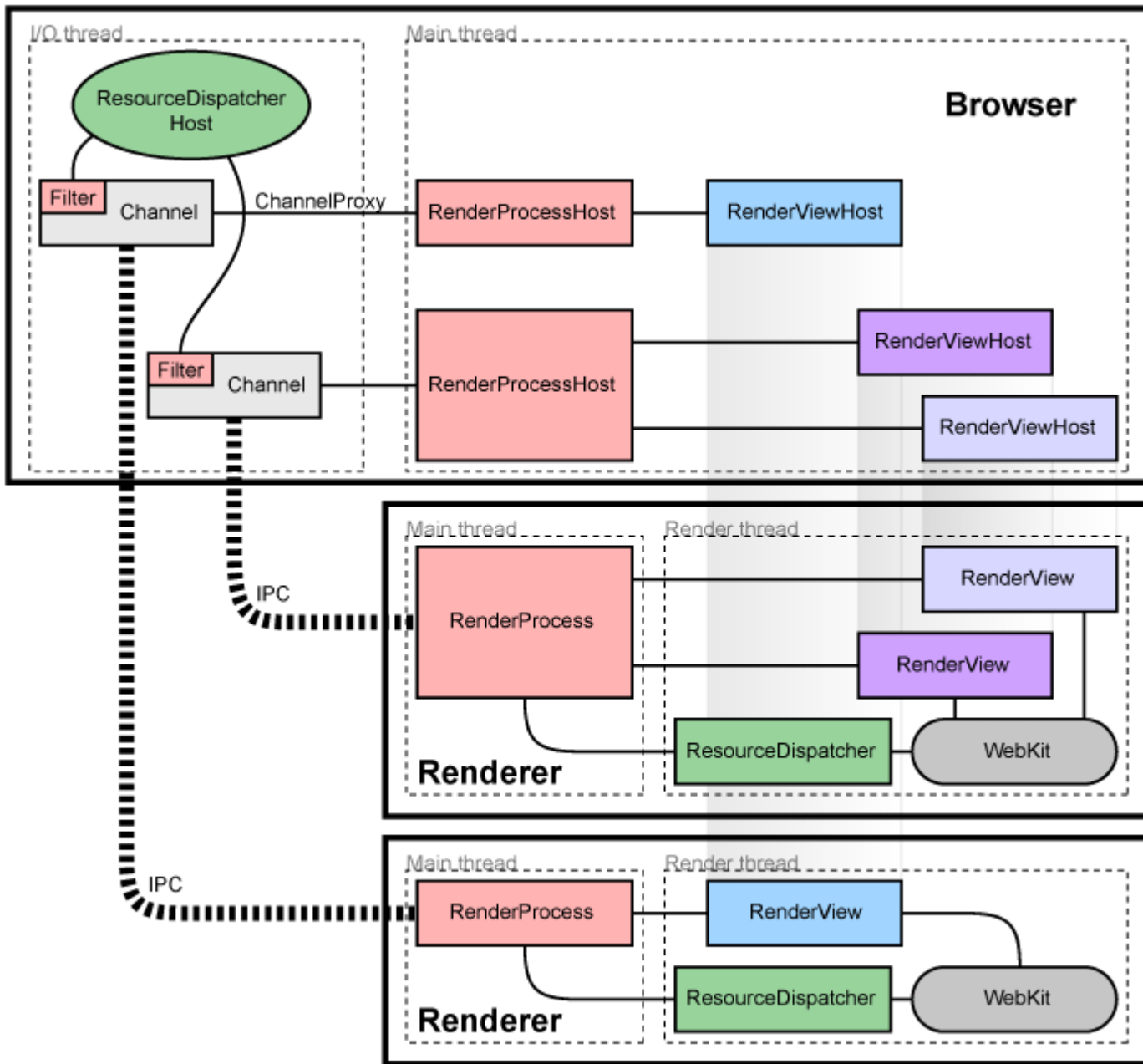
Downloading

Rendering

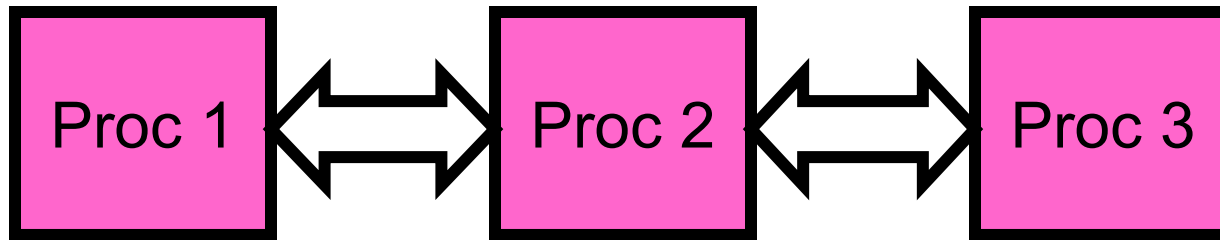
Painting



N



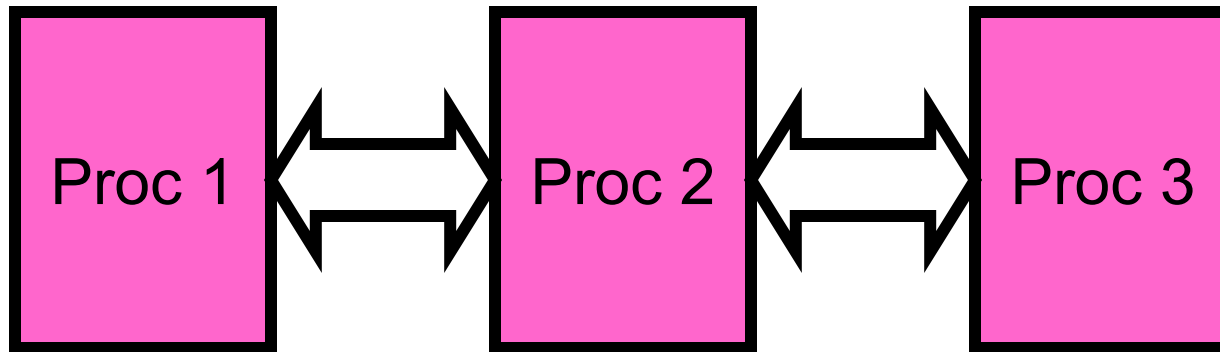
Cooperating Processes



- Multiple processes collaborate on a task
- Advantages of process cooperation
 - Information sharing
 - Computation speed-up
 - Modularity
 - Convenience

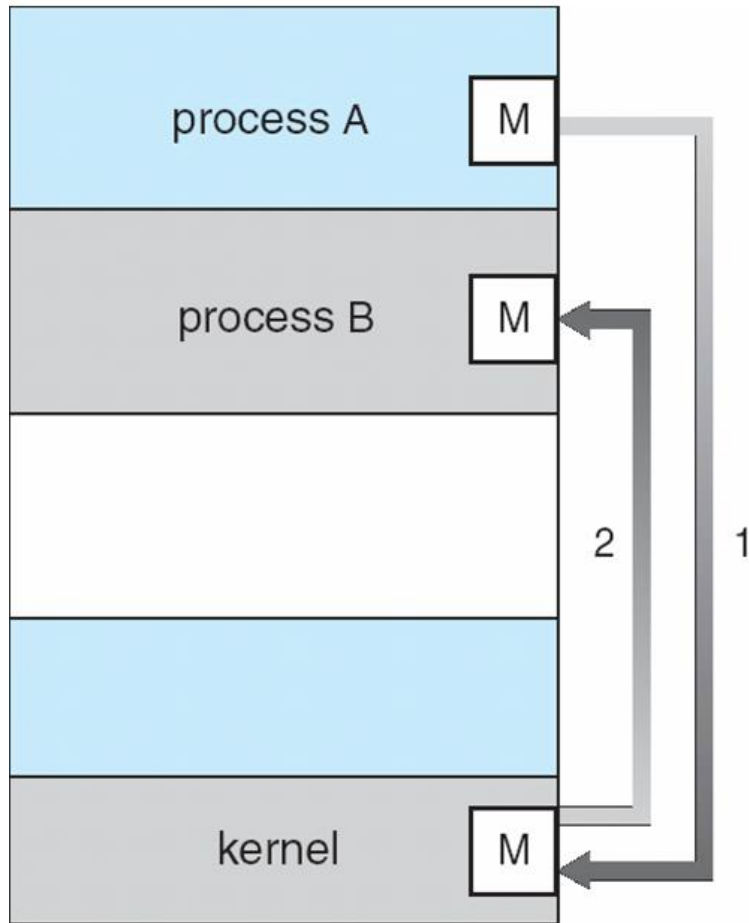


Cooperating Processes

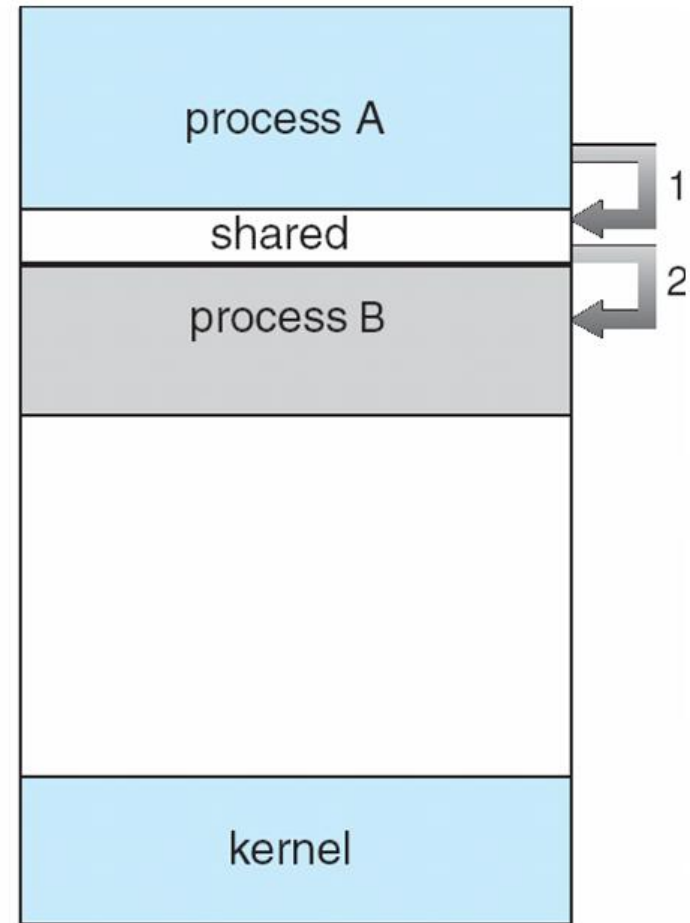


- Need Communication mechanism:
 - Separate address spaces isolate processes
 - **Shared-Memory Mapping**
 - Accomplished by mapping addresses to common DRAM
 - Read and Write through memory
 - **Message Passing**
 - `send()` and `receive()` messages
 - Works across network

Communication Models



(a)



(b)

Examples of IPC Systems - POSIX

- **POSIX Shared Memory**

- Process A first creates shared memory segment

```
segment_id = shmget(IPC_PRIVATE, size,  
S_IRUSR | S_IWUSR);
```

- Process B wanting access to that shared memory must attach to it

```
shared_memory = (char *) shmat(id, NULL, 0);
```

- Now the process B could write to the shared memory

```
sprintf(shared_memory, "Writing to shared  
memory");
```

- When done a process can detach the shared memory from its address space

```
shmdt(shared_memory);
```

Why Processes & Threads?

Goals:

- Multiprogramming: Run multiple applications concurrently
- Protection: Don't want a bad application to crash system!



Solution:

Process: unit of execution and allocation

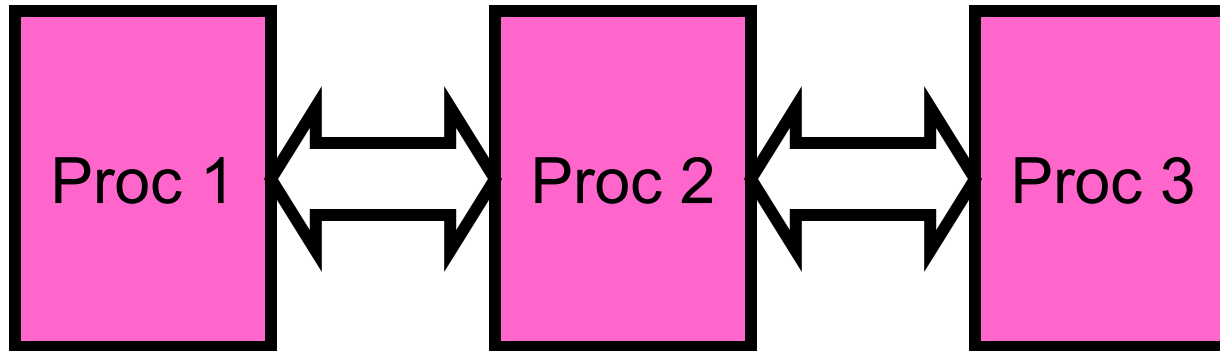
- Virtual Machine abstraction: give process illusion it owns machine (i.e., CPU, Memory, and IO device multiplexing)



Challenge:

- Process creation & switching expensive
- Need concurrency within same app (e.g., web server)

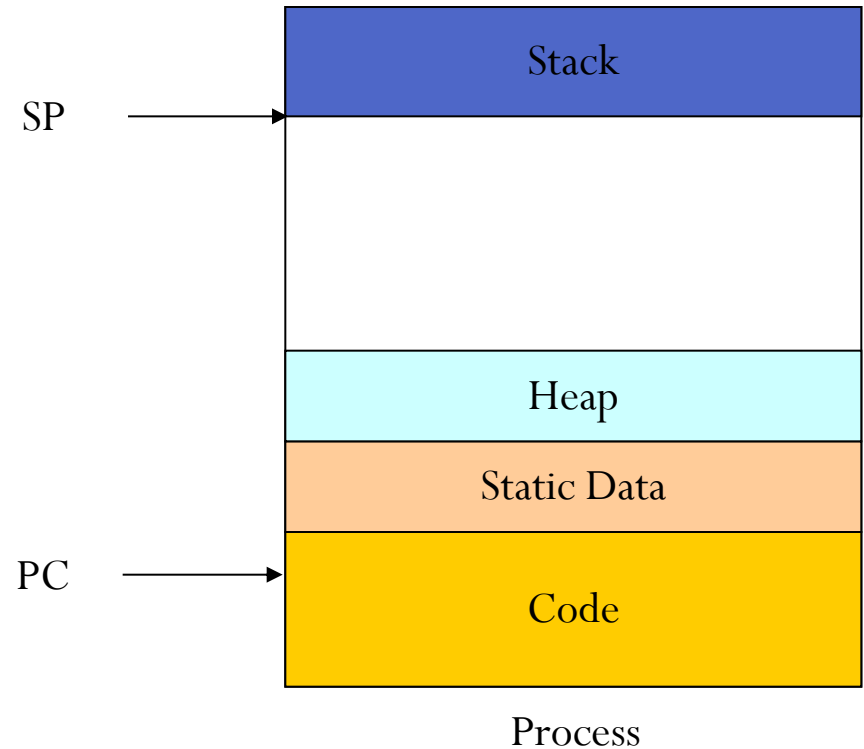
Cooperating Processes



- Need Communication mechanism:
 - Separate address spaces isolate processes
 - Shared-Memory Mapping
 - Message Passing
- Expensive (i.e., context switches)

Memory Structure for Process

- Execution context
 - Program counter (PC)
 - Stack pointer (SP)
 - Data registers
- Code
- Data
- Stack



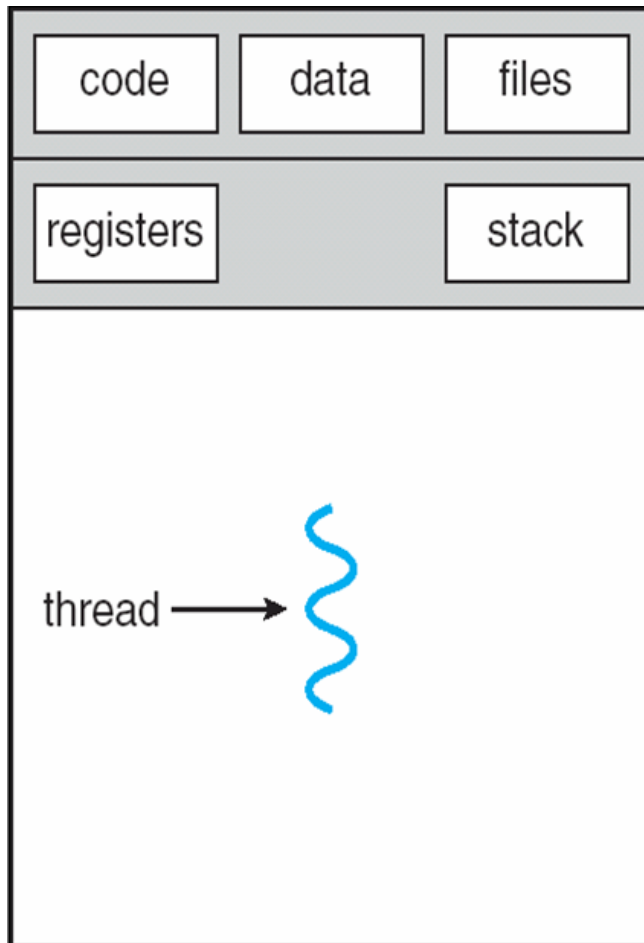
This is how technology evolves...

- we have some problems with inter-process collaboration
 - (this requires a fundamental change) How about the concept of thread?
 - does the concept of thread work with the traditional architecture of process?

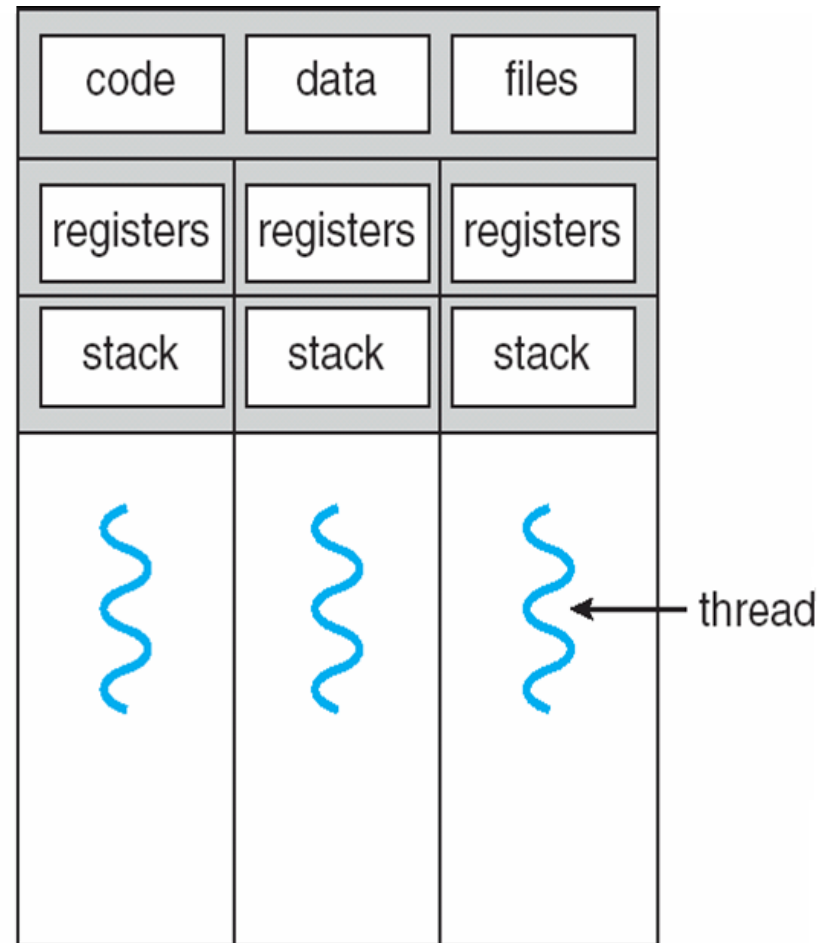
Thread States

- ✂ Threads should do context switches, as processes do, but it must be in a bit different way
- State shared by all threads in process/addr space
 - Content of memory (global variables, heap)
 - I/O state (file system, network connections, etc)
- State “private” to each thread
 - Kept in TCB $\equiv$ Thread Control Block
 - CPU registers (including, program counter)
 - Execution stack – what is this?
- Execution Stack
 - Parameters, temporary variables
 - Return PCs are kept while called procedures are executing

Threads



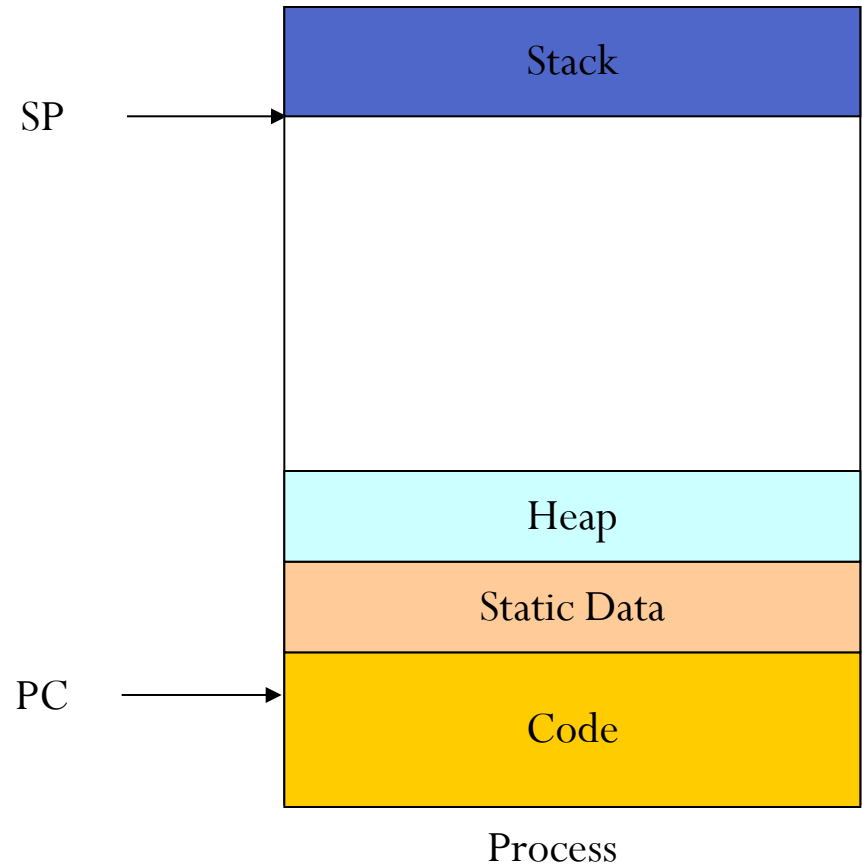
single-threaded process



multithreaded process

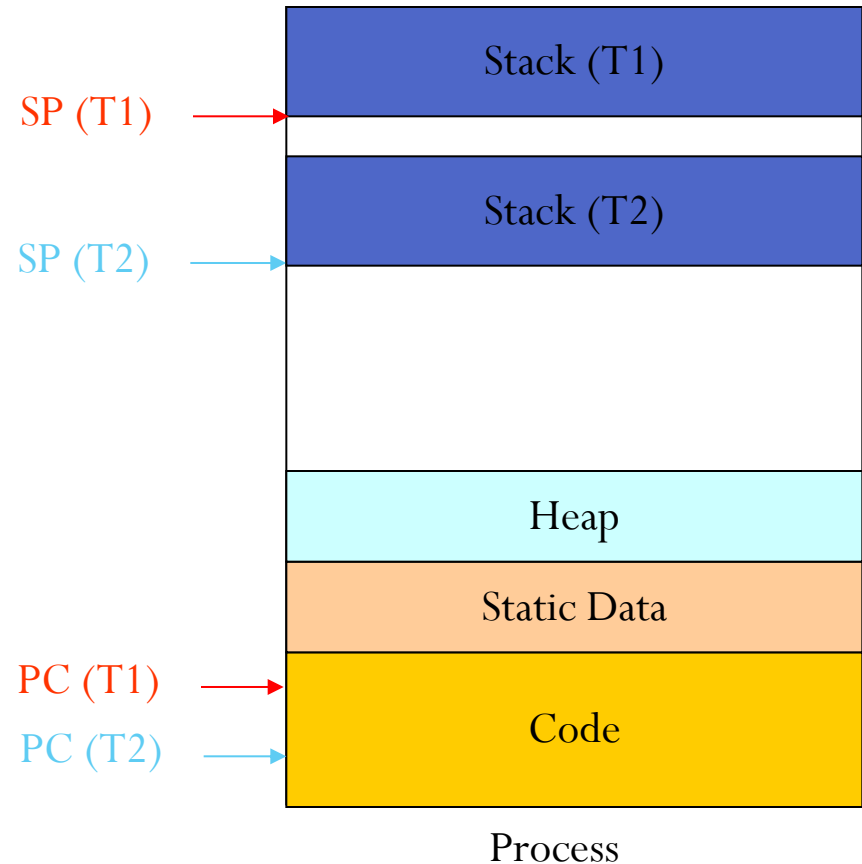
Process vs. Thread

- Execution context
 - Program counter (PC)
 - Stack pointer (SP)
 - Data registers
- Code
- Data
- Stack



Process vs. Thread

- Execution context
 - Program counter (PC)
 - Stack pointer (SP)
 - Data registers
- Stack

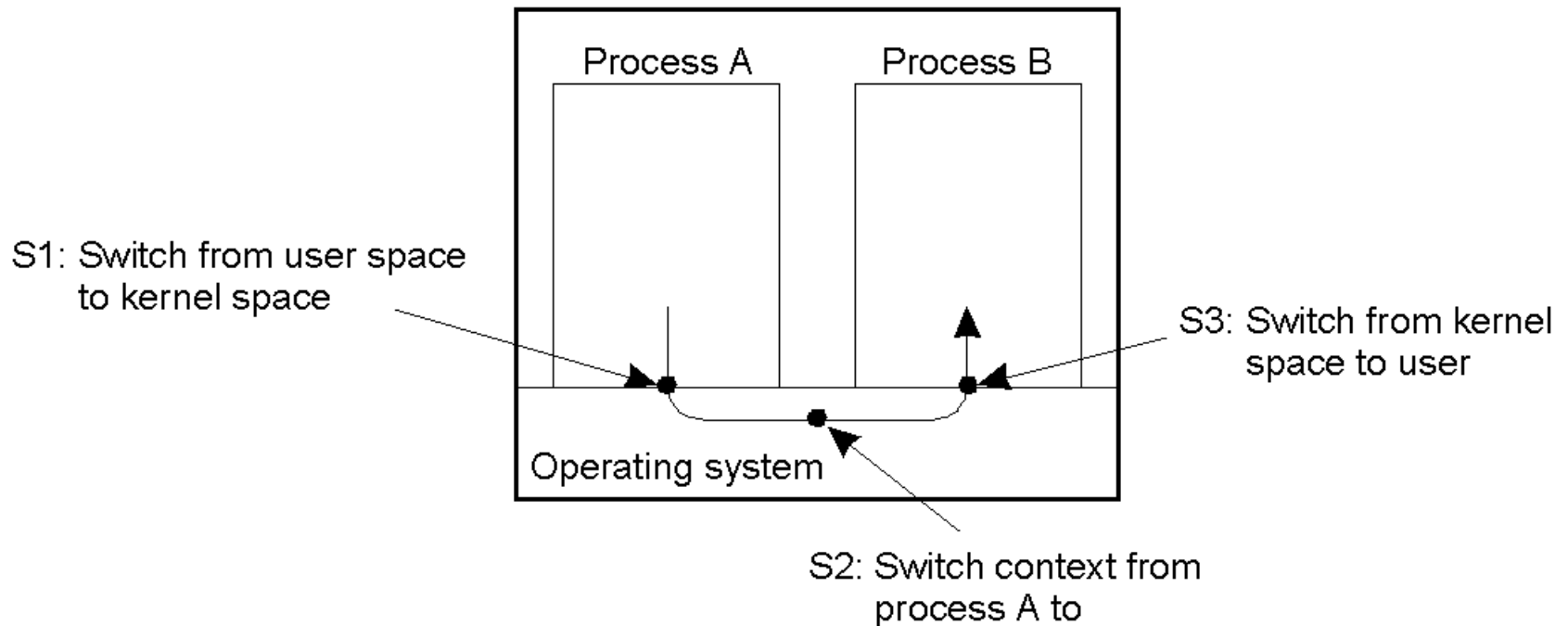


Process vs. Thread (1)

- A thread is bounded to a single process
- A process can have multiple threads
- Sharing data between threads is cheap: all see the same address space
- Threads become the unit of scheduling
- Processes are now containers in which threads execute

Process vs. Thread (2)

- Processes
 - Inter-process communication is expensive: need to perform context switch
 - Secure: one process cannot corrupt another process



Process vs. Thread (3)

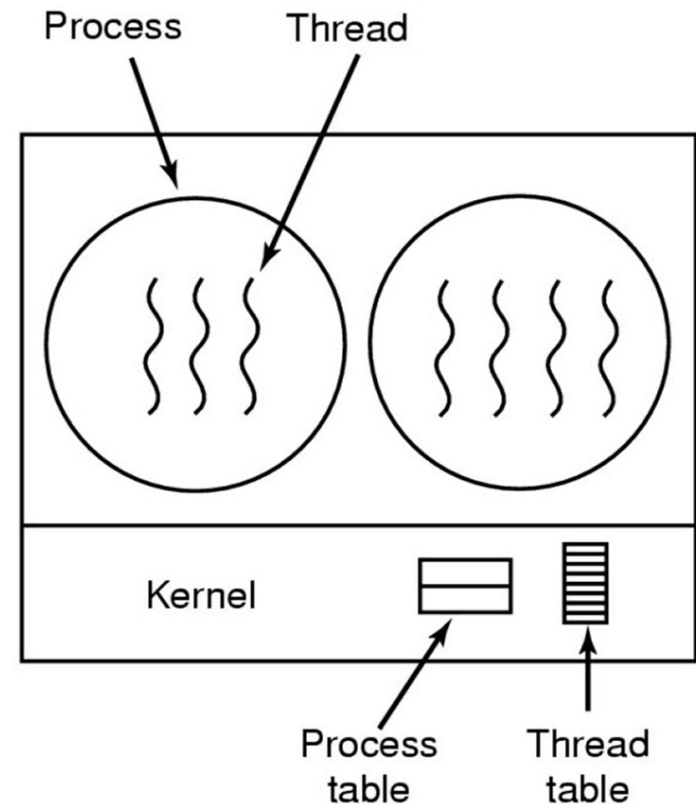
- Threads
 - Inter-thread communication cheap: can use process memory and may not need to context switch
 - Not secure: a thread can write the memory used by another thread

Threading Issues

- Who takes care of thread management?
 - OS (kernel threads)
 - System calls for thread creation and management
 - User-level process (user-level threads)
 - a library linked into the program takes care of it

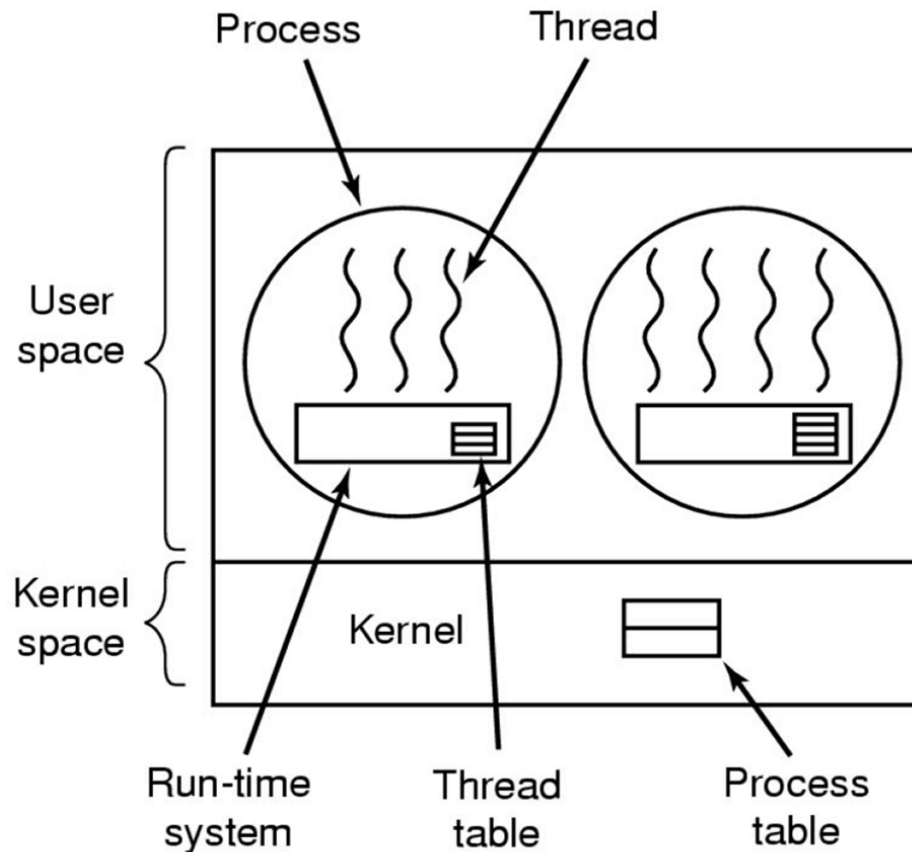
Kernel-level Threads

- OS takes care of all thread operations, including creation, management, scheduling, & context switch
- Similar to process impl.
 - System-call base
 - TCB : thread control block



User-level Thread Management

- User-level library takes care of all thread operations, including creation, management, scheduling, & context switch



User-level Thread Management

- User-level library takes care of all thread operations, including creation, management, scheduling, & context switch
- How is it possible?
 - Threads share the same address space
 - The thread manager does not need to manipulate address space
 - Threads only differ in hardware contexts (PC, SP, registers) that can be manipulated by user-level process itself

User-level Threads

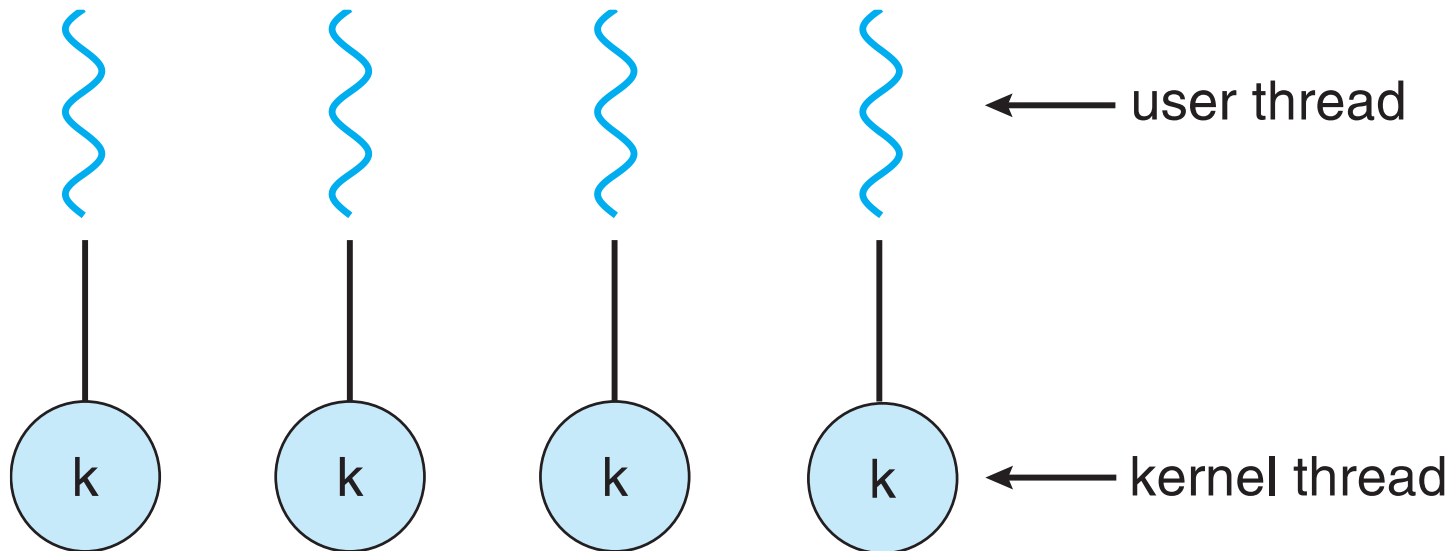
- Efficient thread operations
 - User library handles PC, registers, stack, TCB
 - Done by procedure calls (no kernel involvement)
 - 10-100x faster than kernel-level thread operations, including context switches
- Limitations
 - Transparent to OS (not well integrated with OS)
 - OS can make a poor scheduling decision
 - can block a process whose thread initiated I/O, even though the process has other runnable threads
 - can unschedule a process with a thread holding a lock

Threading Models

- How to map user-level threads to kernel threads
 - Many-to-one (N:1)
 - One-to-one (1:1)
 - Many-to-many (M:N)

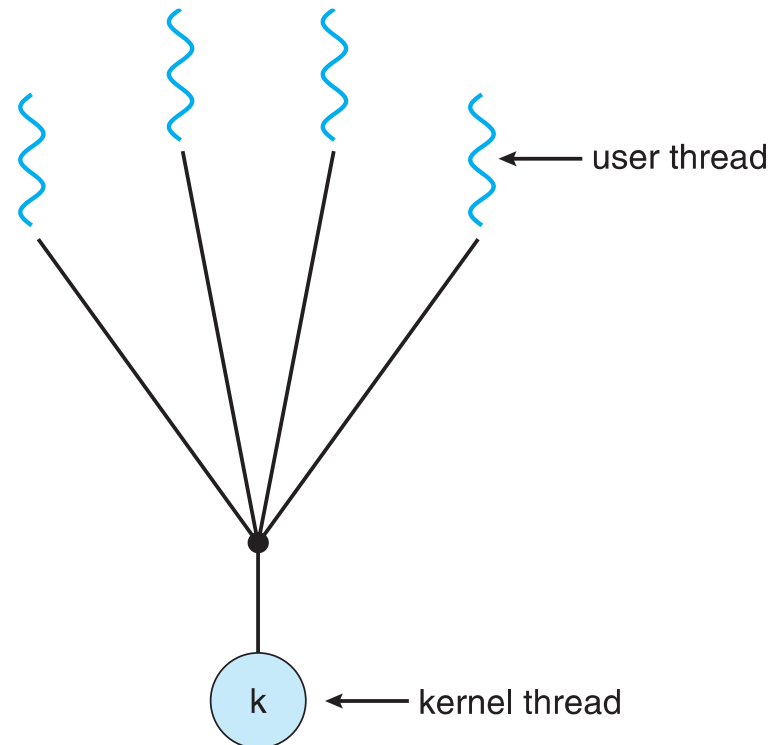
Threading Models

- One-to-One (1:1)
 - Each user-level thread mapped to a single kernel thread
 - Overhead of creating kernel threads can burden performance
 - Examples
 - Windows 10
 - Linux
 - Solaris 9 and later



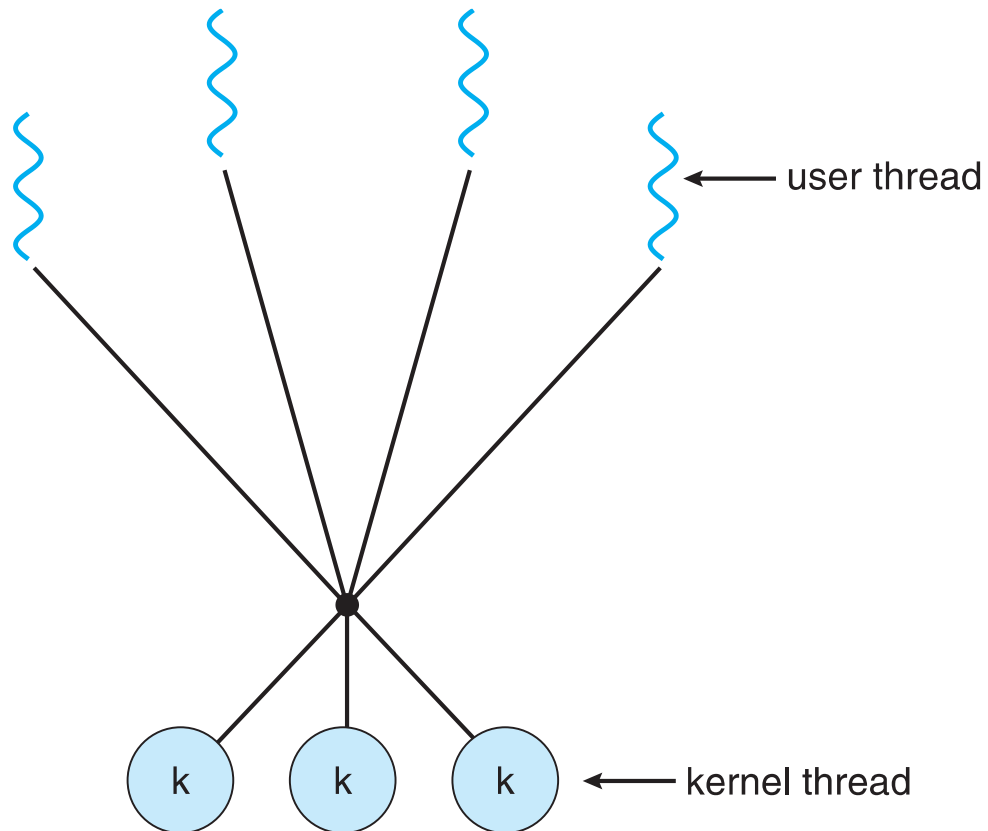
Threading Models

- Many-to-One (N:1)
 - Many user-level threads mapped to a single kernel thread
 - Used on systems that do not support kernel threads
 - Efficient, but may block an entire process with a thread on blocking; unable to run on multiprocessors
 - Examples
 - Solaris Green Threads
 - GNU Portable Threads



Threading Models

- Many-to-Many (M:N)
 - Many user-level threads mapped to many kernel threads
 - Allows OS to create a sufficient number of kernel threads
 - Examples
 - Solaris 2
 - IRIX



Fundamental OS Design Issue

- Who takes care of some features?
 - OS
 - Allows global viewpoints
 - User-level library
 - Can be more efficient for local actions (thru function calls rather than system calls)